CHƯƠNG 8

Xây dựng ứng dụng AI tạo sinh

[Ứng dụng Trí tuệ nhân tạo]



[Nguyễn Tuấn Anh]
• [Thái Nguyên, 2026]

Giới thiệu nội dung

Mục tiêu chương

- Mô tả quy trình 5 giai đoạn xây dựng ứng dụng GenAI từ ý tưởng đến sản phẩm.
- Phân biệt & lựa chọn ba mô hình tích hợp AI phù hợp với từng bài toán.
- Xây dựng chiến lược kiểm thử toàn diện & cơ chế giảm thiểu hallucination.
- Phát triển dự án GenAI hoàn chỉnh theo quy trình 9 bước, tích hợp kiến thức các chương trước.

Nội dung chính

1. Quy trình xây dựng ứng dụng AI tạo sinh
2. Tích hợp mô hình AI vào ứng dụng
3. Kiểm thử, đánh giá & Phát triển dự án hoàn chỉnh

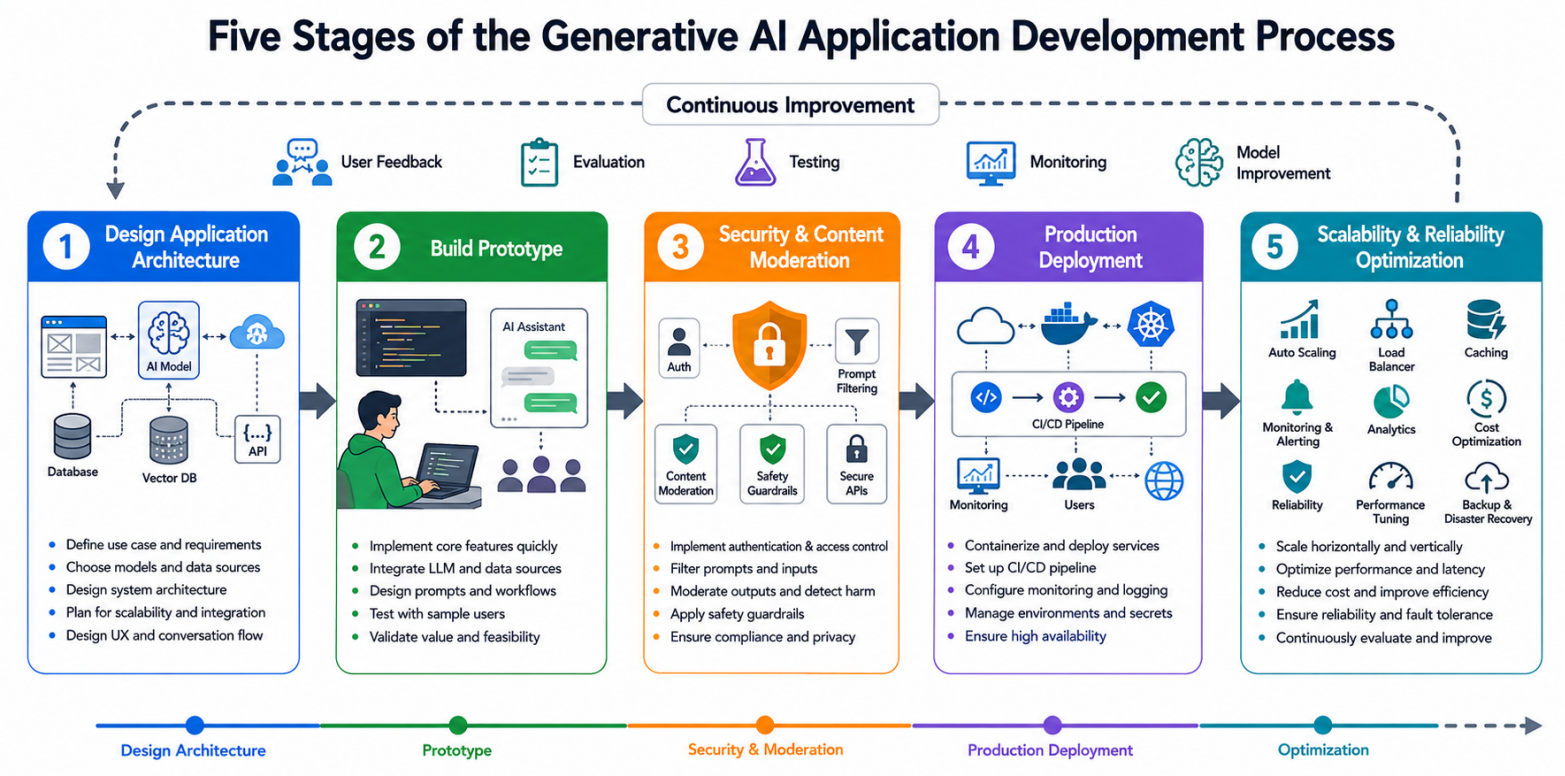
Khởi động: Hành trình khác biệt của ứng dụng GenAI

- Xây dựng trợ lý học tập thông minh: giải thích khái niệm, đưa bài tập, chấm điểm tự luận – **vừa giống, vừa khác** phát triển phần mềm truyền thống.
- **Giống**: vẫn cần phân tích yêu cầu, thiết kế hệ thống, viết code, kiểm thử.
- **Khác**: "trí thông minh" đến từ LLM thay vì từ code; thứ tự ưu tiên và tiêu chí thành công thay đổi đáng kể.

Thực tế đáng chú ý

Nhiều ứng dụng GenAI **không vượt qua được giai đoạn prototype** – không phải vì ý tưởng kém, mà vì đội ngũ đánh giá thấp thách thức về khả năng mở rộng, chi phí vận hành và **tính không xác định** của đầu ra mô hình.

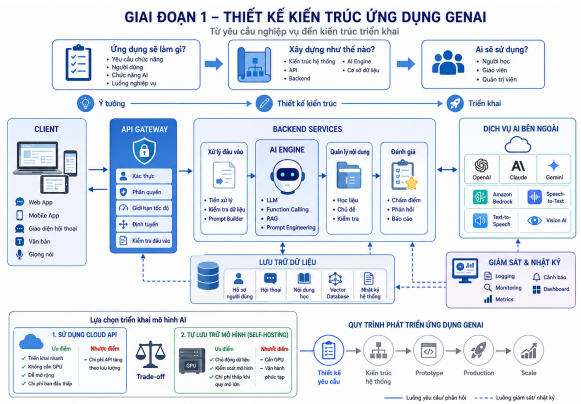
Năm giai đoạn của quy trình xây dựng ứng dụng GenAI



Quy trình không tuyến tính cứng nhắc – thường xuyên quay lại các giai đoạn trước dựa trên kết quả kiểm thử và phản hồi người dùng.

Giai đoạn 1: Thiết kế kiến trúc ứng dụng

- **Ba câu hỏi nền tảng**: Ứng dụng sẽ làm gì? Xây dựng như thế nào? Ai sẽ sử dụng?
- **Yêu cầu chức năng** (Functional): diễn đạt cụ thể, kiểm chứng được. Ví dụ: “người dùng nhận câu trả lời trong ≤ 5 giây” tốt hơn “ứng dụng trả lời câu hỏi”.
- **Yêu cầu phi chức năng** (Non-functional): xác thực & phân quyền, bảo mật dữ liệu, hiệu năng, khả năng mở rộng, ghi log & giám sát, thông báo – **đặc biệt quan trọng** với GenAI.



Giai đoạn 2 & 3: Prototype và Bảo mật

Giai đoạn 2: Xây dựng prototype

- Mục tiêu: **MVP** – phiên bản chạy được với chức năng tối thiểu để kiểm chứng ý tưởng.
- Nguyên tắc: càng sớm có người dùng thật, càng sớm biết điều gì thực sự có giá trị.
- Giữ kiến trúc **đơn giản** để thay đổi nhanh; đừng tối ưu hóa quá sớm.

Giai đoạn 3: Bảo mật & Kiểm duyệt

- **Prompt injection**: kẻ tấn công chèn chỉ thị độc hại vào input.
- **Data leakage**: mô hình tiết lộ dữ liệu nhạy cảm từ ngữ cảnh.
- **Harmful content**: mô hình sinh nội dung độc hại.

Nguyên tắc: **defense in depth** – không dựa vào một cơ chế bảo vệ duy nhất.

Bốn lớp bảo vệ: Input validation (moderation API) → System prompt

Giai đoạn 4 & 5: Production và Tối ưu hóa

Giai đoạn 4: Triển khai production

- **Container hóa** (Docker): đảm bảo nhất quán mọi môi trường.
- **CI/CD Pipeline**: tự động hóa kiểm thử, build, deploy – **bao gồm cả gọi LLM API với test prompts**.
- **Multi-region deployment**: đảm bảo độ trễ thấp và tính sẵn sàng cao.

Giai đoạn 5: Tối ưu hóa

- **Giám sát hiệu năng mô hình**: thời gian phản hồi, tỉ lệ lỗi API, phân phối độ dài phản hồi.
- **Thu thập phản hồi**: thumbs up/down, xếp hạng sao – nguồn dữ liệu quý giá.
- **Cải thiện liên tục**: điều chỉnh prompt, cập nhật tri thức, thử nghiệm model mới.

Build – Measure – Learn: Quy trình lặp

- **Khác biệt cốt lõi** với phát triển truyền thống: cùng một prompt có thể tạo ra các phản hồi khác nhau – không phải lúc nào cũng biết trước đâu là "đúng".
- Quy trình phát triển GenAI cần theo mô hình **lặp** thay vì thác nước (waterfall).

1. **Build**: xây dựng phiên bản với **giả thuyết rõ ràng** về điều sẽ tạo giá trị.
2. **Measure**: đo lường hành vi thực tế – cả chỉ số kỹ thuật (độ trễ, tỉ lệ lỗi) và chất lượng (độ chính xác, mức độ hài lòng).
3. **Learn**: rút bài học từ dữ liệu, điều chỉnh giả thuyết, quyết định bước tiếp theo – cải thiện prompt, đổi model, hoặc pivot.

Nguyên tắc vàng

Một tuần với **ba vòng** Build-Measure-Learn nhỏ có giá trị hơn một tháng xây dựng không có phản hồi.

Case Study: Hành trình xây dựng SkillGenie (1/2)

Bài toán: Người học lập trình gặp khó khăn: không ai giải thích khái niệm khó, không biết đang hiểu sai ở đâu, thiếu lộ trình cá nhân hóa.
Các bước đã thực hiện:

1. **Phát biểu bài toán**: xác định vấn đề và đề xuất giá trị cốt lõi.
2. **Thiết kế UX**: 5 bước user journey (chào mừng, chọn chế độ, tương tác, đánh giá, theo dõi).
3. **Thiết kế hệ thống**: client-server, API Gateway, 4 backend services, 3 tầng lưu trữ.
4. **Xây dựng prototype**: 1 chủ đề, chế độ tự học, OpenAI API, 10 người dùng.

Case Study: Hành trình xây dựng SkillGenie (2/2)

5. **Học từ phản hồi**: Người dùng thích chất lượng giải thích, nhưng không biết hỏi gì tiếp theo. $\rightarrow$ Thêm "câu hỏi gợi ý" – tỉ lệ tương tác **tăng 40%**.
6. **Tiến tới production**: 3 vòng Build-Measure-Learn; triển khai đầy đủ các lớp bảo mật; Docker; CI/CD; giám sát liên tục.

Bài học từ SkillGenie

Insight quan trọng nhất đến từ việc **cho người dùng thật trải nghiệm sớm** – không phải từ phân tích lý thuyết. Prototype càng đơn giản, bạn càng dám thay đổi khi phát hiện sai hướng.

Tổng kết Section 1 – Tự kiểm tra

Những điểm cần nhớ

1. **5 giai đoạn**: Thiết kế → Prototype → Bảo mật → Production → Tối ưu hóa. Không tuyến tính.
2. **Yêu cầu phi chức năng** (bảo mật, hiệu năng, khả năng mở rộng) đặc biệt quan trọng với GenAI.
3. **Build – Measure – Learn**: mô hình lặp bắt buộc vì tính không xác định của LLM.
4. **Defense in depth**: không dựa vào một cơ chế bảo vệ duy nhất.

Tự kiểm tra nhanh:

1. Liệt kê 5 giai đoạn. Vì sao giai đoạn 3 (bảo mật) đặc biệt quan trọng với GenAI?
2. Tại sao quy trình GenAI cần hướng lặp thay vì tuyến tính?

Ba mô hình tích hợp AI: Từ đơn giản đến phức tạp

- Làm thế nào để ứng dụng của bạn "nói chuyện" được với LLM trên đám mây? Các nhà cung cấp đã đóng gói sức mạnh mô hình đằng sau **REST API** quen thuộc.
- **Khác biệt**: response không được xác định trước bởi code – được sinh ra bởi mô hình tại thời điểm request.

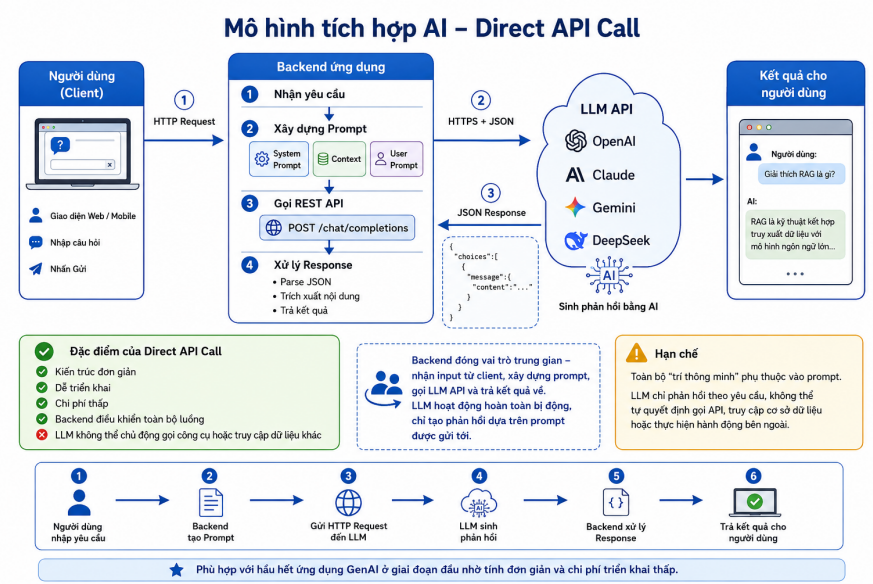
1. **Direct API Call**: Gửi prompt, nhận text – đơn giản nhất, phổ biến nhất.
2. **Function Calling**: Model **đề xuất** gọi hàm (sinh JSON), application code **thực thi**.
3. **Agent-based Integration**: Agent tự lập kế hoạch, chọn công cụ, thực hiện nhiều vòng lặp suy nghĩ – hành động – quan sát.

Nguyên tắc chọn

Luôn bắt đầu với mô hình **đơn giản nhất** có thể giải quyết được bài toán.

Mô hình 1: Direct API Call

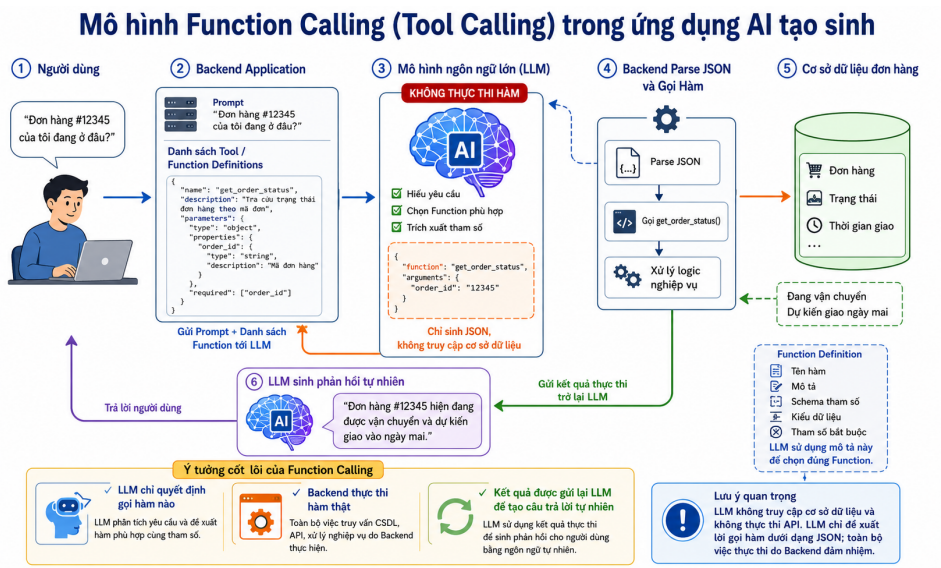
- Backend nhận input → xây dựng prompt → gọi LLM API → trả kết quả.
- **Ưu điểm**: đơn giản, chi phí triển khai thấp, phù hợp phần lớn ứng dụng giai đoạn đầu.
- **Hạn chế**: toàn bộ "trí thông minh" nằm trong prompt – mô hình **không thể** chủ động yêu cầu thêm dữ liệu hoặc gọi chức năng khác.



Backend là trung gian: nhận input, gọi LLM API, trả kết quả.

Mô hình 2: Function Calling

- **Cơ chế**: Model phân tích input → xác định hàm phù hợp → sinh JSON chứa tên hàm + tham số → Application code parse và thực thi → gửi kết quả về model để sinh phản hồi cuối cùng.
- **Điểm then chốt**: Model **không thực thi hàm** – chỉ đề xuất. Application code quyết định có thực thi hay không.



(1) User gửi câu hỏi; (2) Backend gửi prompt + function definitions; (3) LLM trả về JSON; (4) Backend parse + thực thi; (5) Gửi kết quả về LLM để sinh câu trả lời tự nhiên.

Function Calling: Minh họa code

- `functions` định nghĩa theo schema `name`, `description`, `parameters`.
- `function_call="auto"` – model tự quyết định có cần gọi hàm không.
- **Thực thi hàm** diễn ra trong application code, model chỉ đề xuất.

```python
import openai, json

functions = [{
    "name": "get_order_status",
    "description": "Tra cứu trạng thái đơn hàng theo mã đơn",
    "parameters": {"type": "object",
        "properties": {"order_id": {"type": "string",
            "description": "Mã đơn hàng, ví dụ: ORD-12345"}},
        "required": ["order_id"]}
}]

response = openai.chat.completions.create(
    model="gpt-4o",
    messages=[{"role": "user", "content": user_message}],
    functions=functions, function_call="auto")
choice = response.choices[0]
if choice.finish_reason == "function_call":
    args = json.loads(choice.message.function_call.arguments)
    result = lookup_order_in_database(args["order_id"])  # App thực thi
```

Mô hình 3: Agent-based Integration

- **Khi nào cần**: ứng dụng phức tạp – nhiều công cụ, nhiều bước xử lý, nhu cầu ra quyết định động.
- **AI Agent** có khả năng: phân tích input & xác định mục tiêu; lập kế hoạch chia thành các bước con; lựa chọn & sử dụng công cụ; đánh giá kết quả trung gian & điều chỉnh; tổng hợp kết quả cuối cùng.

Kiến trúc Agent:

- **Agent Controller**: điều phối luồng suy nghĩ – hành động.
- **Tool Registry**: danh sách các công cụ agent có thể sử dụng.
- **Memory Manager**: quản lý ngữ cảnh ngắn hạn và dài hạn.
- **Execution Engine**: thực thi các hành động được agent đề xuất.

Lưu ý

Agent có thể chọn hướng đi không mong đợi – khả năng kiểm soát thấp hơn so với Direct API Call và Function Calling.

So sánh ba mô hình tích hợp AI

Tiêu chí	Direct API Call	Function Calling	Agent-based
Độ phức tạp	Thấp	Trung bình	Cao
Tương tác hệ thống	Không	Có – model đề xuất, app thực thi	Mạnh – phối hợp n công cụ
Tính tự chủ	Không (bị động)	Thấp (chỉ đề xuất)	Cao (tự lập kế hoạ
Chi phí vận hành	Thấp (1 lần gọi)	Trung bình (2-3 lần)	Cao (nhiều vòng lặ
Kiểm soát	Cao	Cao	Trung bình
Tình huống phù hợp	Q&A, sinh nội dung, tóm tắt	Tra cứu dữ liệu, CRUD, tích hợp API	Tác vụ nhiều bước lý thông minh

Nguyên tắc: luôn bắt đầu với Direct API Call. Chỉ nâng cấp lên Function Calling hoặc Agent khi đơn giản không đủ.

Thiết kế API cho ứng dụng GenAI

Cấu trúc Request & Response:

- Request: `system_context`, `messages`, `params`.
- Response: `content` + `finish_reason` + `usage` + `metadata`.

Xử lý lỗi (Error Handling):

- Rate Limit (429): exponential backoff + jitter.
- Server Error (5xx): retry ≤ 3 lần, circuit breaker.
- Content Filter: không retry – thông báo diễn đạt lại.
- Invalid JSON: retry 2 lần rồi fallback về text

Bảo vệ API endpoint:

- API key $\rightarrow$ environment variables / secrets manager.
- Giới hạn độ dài input; ngân sách token/ngày.
- Luôn lọc input + output qua moderation API.

API endpoint = mục tiêu tấn công $\rightarrow$ **defense in depth**.

Chat Completions API vs Assistants API

Tiêu chí	Chat Completions API	Assistants API
Quản lý context	Developer tự quản lý – gửi toàn bộ messages mỗi request	OpenAI quản lý – context tự động qua threads
Công cụ tích hợp	Function calling (developer định nghĩa)	Code Interpreter, File Search, Functions (tích hợp sẵn)
Độ phức tạp	Đơn giản	Phức tạp hơn (assistant, thread, run lifecycle)
Kiểm soát	Cao – toàn quyền quyết định context	Trung bình – context do OpenAI quản lý
Tình huống phù hợp	Chatbot đơn giản, sinh nội dung, Q&A một lượt	Trợ lý cá nhân, phân tích tài liệu, hội thoại dài

Chat Completions API: linh hoạt, kiểm soát cao. **Assistants API**: tiện lợi cho hội thoại dài & xử lý file, ít kiểm soát hơn.

Tổng kết Section 2 – Tự kiểm tra

Những điểm cần nhớ

1. **3 mô hình tích hợp**: Direct API Call → Function Calling → Agent-based. Chọn từ đơn giản nhất.
2. **Function Calling**: Model **không thực thi hàm** – chỉ đề xuất qua JSON. Đây là quyết định thiết kế quan trọng vì bảo mật và kiểm soát.
3. **API Gateway** là trung tâm: xác thực, rate limiting, routing. Response cần finish_reason, usage, metadata.
4. **Xử lý lỗi**: exponential backoff + jitter cho rate limit; không retry với content filter; circuit breaker cho server error.

Tự kiểm tra nhanh:

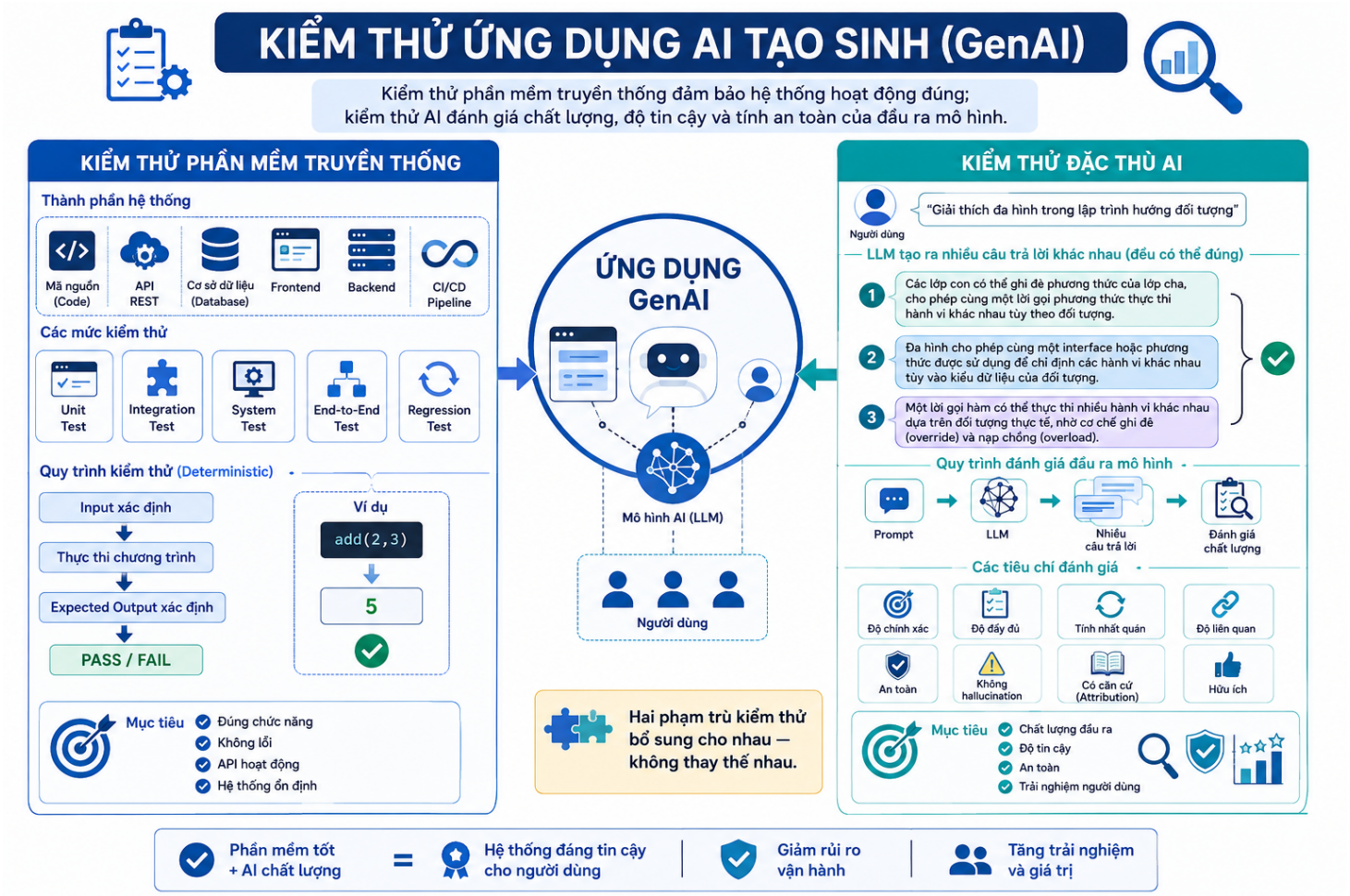
1. Tại sao model không thực thi hàm trong Function Calling lại là quyết định quan trọng?

Kiểm thử GenAI: Bài toán mới

- **Thí nghiệm**: Hỏi cùng một câu hỏi 3 lần $\rightarrow$ 3 câu trả lời khác nhau. Có thể tất cả đều "đúng".
- **Thách thức cốt lõi**: Làm thế nào để kiểm thử một hệ thống mà đầu ra **không được xác định trước**?
- Trong kiểm thử truyền thống: `add(2, 3)` phải trả về `5` – không hơn, không kém.
- Với LLM: "Giải thích khái niệm đa hình" có thể nhận **hàng trăm** câu trả lời khác nhau, tất cả đều có thể "đúng".

Cần một **khung kiểm thử mới** – kết hợp kiểm thử phần mềm truyền thống với các kỹ thuật đánh giá đặc thù cho mô hình ngôn ngữ.

Hai phạm trù kiểm thử cho ứng dụng GenAI



Phạm trù 1: Kiểm thử phần mềm truyền thống

- **Unit Tests**: kiểm tra hàm/module **không liên quan LLM** – xử lý input, parse JSON, quản lý session. Ví dụ: xác nhận hàm `parse_function_call_response()` xử lý đúng cả JSON hợp lệ lẫn JSON lỗi.
- **Integration Tests**: kiểm tra luồng dữ liệu API Gateway $\rightarrow$ Backend; kết nối LLM API với test prompts đơn giản; tương tác giữa các service trong pipeline RAG.
- **End-to-End Tests**: kiểm tra toàn bộ luồng người dùng – gửi câu hỏi, xác nhận response trong thời gian chấp nhận được, kiểm tra response không rỗng và đúng định dạng.

Lưu ý quan trọng

E2E tests cho GenAI **không nên kiểm tra nội dung** của response – vì điều đó thuộc về phạm trù thứ hai.

Phạm trù 2: Kiểm thử đặc thù AI

- **Độ chính xác** (Accuracy): câu trả lời có đúng về mặt thực tế? Có grounded không?
- **Độ phù hợp** (Relevance): có liên quan đến câu hỏi không? Có đi lạc đề không?
- **Tính nhất quán** (Consistency): câu hỏi tương tự có câu trả lời nhất quán không?
- **An toàn** (Safety): có chứa nội dung độc hại, phân biệt không?
- **Hallucination**: có bịa đặt thông tin không có trong dữ liệu nguồn không?
- **Thiên kiến** (Bias): có thể hiện thiên kiến về giới tính, chủng tộc không?

Các chỉ số này **không thể** đo bằng assertion đơn giản như kiểm thử truyền thống – cần phương pháp đánh giá chuyên biệt.

Thách thức kiểm thử đầu ra không xác định

Tính không xác định (non-determinism) xuất phát từ hai nguồn:

1. Quá trình sinh văn bản là **xác suất**: cùng input $\rightarrow$ output khác nhau qua các lần chạy.
2. Nhà cung cấp LLM có thể **cập nhật phiên bản mô hình** phía sau API mà không thông báo.

Ba chiến lược đối phó:

1. **Criterion-based evaluation**: định nghĩa tập tiêu chí, đánh giá response có đáp ứng từng tiêu chí không (thay vì so với một "đáp án đúng" duy nhất).
2. **LLM-as-judge**: dùng LLM mạnh hơn để đánh giá chất lượng đầu ra – mở rộng tốt nhưng cần kiểm chứng chéo.
3. **Human evaluation**: tiêu chuẩn vàng cho tiêu chí định tính (tự nhiên, giọng điệu, hữu ích) – tốn kém, khó mở rộng, dùng cho cột mốc quan trọng.

Các chỉ số đánh giá chất lượng mô hình

Chỉ số	Tên đầy đủ	Áp dụng cho	Cách hoạt động
ROUGE	Recall-Oriented Understudy for Gisting Evaluation	Tóm tắt văn bản	So sánh độ chồng lấp văn bản sinh và than
BLEU	Bilingual Evaluation Understudy	Dịch máy	So sánh n-gram + br nếu bản dịch quá ng
BERTScore	BERT-based Similarity Score	Đa dạng tác vụ sinh văn bản	Dùng embedding tương đồng **ngữ ngh** vựng
HELM	Holistic Evaluation of Language Models	Đánh giá toàn diện LLM	Benchmark đa chiề calibration, robustne bias, toxicity

Hạn chế của chỉ số tự động

ROUGE/BLEU chỉ đo tương đồng **bề mặt** (từ vựng). Một câu trả lời hoàn toàn chính xác nhưng dùng từ khác bản tham chiếu $\rightarrow$ điểm thấp. Một câu sai nhưng dùng nhiều từ giống $\rightarrow$ điểm cao. Chỉ nên dùng làm **công cụ sàng lọc ban đầu**.

Hallucination: Nhận diện và giảm thiểu

- **Hallucination**: mô hình sinh thông tin **không có thật** nhưng trình bày **tự tin** như sự thật – bịa số liệu, trích dẫn giả, hàm API không tồn tại.

Ba nguyên nhân chính:

1. **Thiếu hụt kiến thức**: ngoài training data – mô hình "điền vào chỗ trống" bằng suy đoán.
2. **Nội suy sai**: kết hợp sai các mảnh thông tin từ nhiều nguồn.
3. **Áp lực tạo câu trả lời**: mô hình không được dạy cách nói "Tôi không biết".

Chiến lược giảm thiểu:

- **Grounding với RAG**: cung cấp tài liệu tham khảo, yêu cầu chỉ trả lời dựa trên tài liệu đó – **hiệu quả nhất**.
- **Contextual grounding check**: tự động kiểm tra từng khẳng định có trong nguồn không.
- **Prompt khuyến khích trung thực + giảm temperature** (gần 0) để output nhất quán hơn.

Quy trình đánh giá ứng dụng GenAI (5 bước)

1. **Xác định chỉ số đánh giá**: chọn chỉ số phù hợp mục tiêu – ứng dụng hỏi đáp: accuracy, faithfulness, hallucination rate; ứng dụng sáng tạo: tính độc đáo, mạch lạc.
2. **Tạo tập dữ liệu kiểm thử**: happy path, edge cases, out-of-scope (kiểm tra từ chối), adversarial (kiểm tra an toàn). Cập nhật định kỳ.
3. **Chạy đánh giá**: chỉ số tự động (ROUGE, BLEU); LLM-as-judge (dùng model đánh giá riêng); human evaluation (phiên định kỳ).
4. **Phân tích kết quả**: theo loại câu hỏi, chủ đề, nhóm người dùng. Điểm tổng thể 85% nhưng một chủ đề chỉ 50% $\rightarrow$ vấn đề cần khắc phục.
5. **Cải thiện**: điều chỉnh prompt, cập nhật tài liệu, thay đổi chiến lược chunking, hoặc chuyển model mạnh hơn. Sau đó **chạy lại đánh giá** và so sánh.

Hiệu năng & AI hỗ trợ kiểm thử

Chỉ số hiệu năng chính:

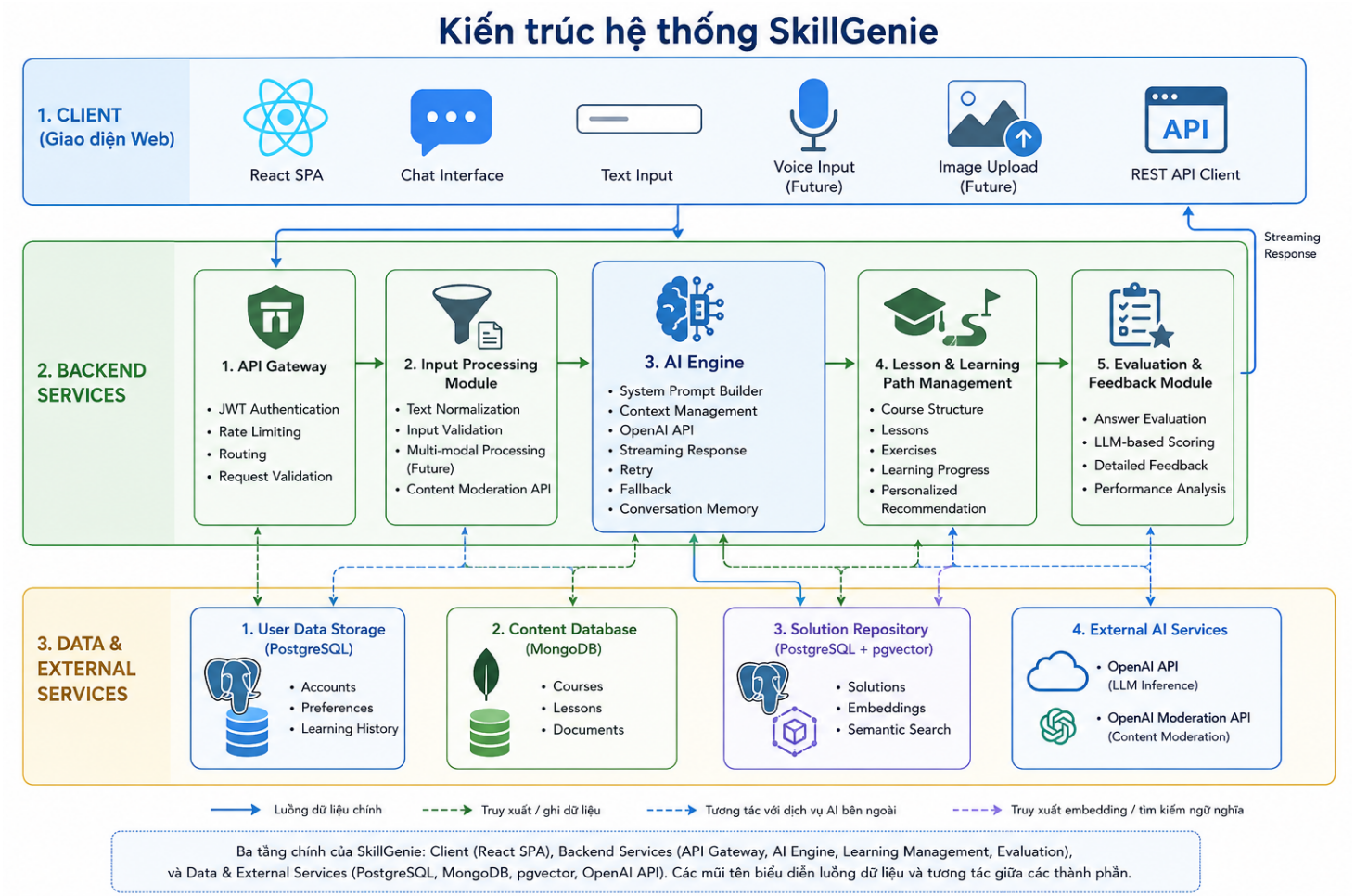
- **TTFT** (Time to First Token): $<$ 1 giây cho ứng dụng chat.
- **Token Generation Rate**: token/giây sau token đầu.
- **End-to-End Latency**: tổng thời gian đến khi nhận câu trả lời hoàn chỉnh.
- **Throughput**: số request đồng thời không suy giảm chất lượng.
- **API Error Rate**: duy trì $<$ 1%.

AI hỗ trợ kiểm thử:

- Tự động sinh unit tests (boundary cases, failure scenarios).
- Quét codebase tìm phương thức chưa được kiểm thử.
- Phân tích error logs, xác định module gây lỗi.
- Đề xuất test scripts cho endpoint mới.

AI không thay thế con người – chỉ hỗ trợ để tập trung vào phán đoán và ngữ cảnh.

Dự án hoàn chỉnh: Kiến trúc hệ thống SkillGenie



Quy trình 9 bước phát triển dự án GenAI (1/2)

1. **Phát biểu bài toán & xác định phạm vi**: Problem statement rõ ràng; phân định trong/ngoài phạm vi phiên bản đầu – tránh scope creep.
2. **Thiết kế trải nghiệm người dùng**: User journey với các bước cụ thể; cơ sở để ước lượng effort.
3. **Thiết kế kiến trúc hệ thống**: Client (SPA), Backend Services (API Gateway, Input Processing, AI Engine, Evaluation), Data & External Services.
4. **Thiết kế API**: "Hợp đồng" frontend-backend; streaming response với SSE cho trải nghiệm mượt mà.
5. **Xây dựng prototype**: Phiên bản đơn giản nhất để kiểm chứng giả thuyết giá trị cốt lõi. **Không authentication, không database, không streaming** ở vòng đầu.

Quy trình 9 bước phát triển dự án GenAI (2/2)

6. **Tích hợp thành phần nâng cao**: RAG (kho tri thức + vector DB); Prompt có cấu trúc (phân tầng: system + template); Human-in-the-loop (AI đánh giá sơ bộ, giáo viên xem xét).
7. **Triển khai bảo mật & kiểm duyệt**: Moderation API; System prompt hardening; Output filtering; Mã hóa dữ liệu; Quản lý API keys.
8. **Triển khai production**: Docker containerization; CI/CD pipeline (GitHub Actions); Monitoring (kỹ thuật + sản phẩm + chất lượng AI); Rolling update, auto-rollback.
9. **Cải thiện liên tục sau triển khai**: Phân tích thumbs down $\rightarrow$ cải thiện prompt; A/B test; Đánh giá chi phí token; Thử nghiệm model mới với test suite.

Triển khai production **không phải đích đến** – là **điểm khởi đầu** cho quá trình cải thiện liên tục.

Bài học từ dự án thực tế

1. **Bắt đầu nhỏ, mở rộng nhanh**: Xác định giả thuyết giá trị cốt lõi $\rightarrow$ MVP $\rightarrow$ kiểm chứng $\rightarrow$ chỉ đầu tư tính năng bổ sung sau khi xác nhận.
2. **Đo lường mọi thứ**: Trong GenAI, không thể dựa vào trực giác. Cần dữ liệu: thumbs up/down, thời gian tương tác, tỉ lệ quay lại, phân tích lỗi.
3. **Thiết kế để thay đổi**: Prompt, model, provider có thể thay đổi. Kiến trúc phải cho phép thay prompt không cần deploy lại; đổi provider chỉ ảnh hưởng một module.
4. **Bảo mật không phải bước cuối cùng**: Ngay từ prototype đã cần moderation API và system prompt cơ bản.
5. **Con người luôn trong vòng lặp**: Với ứng dụng tác động lớn (như đánh giá học tập), không để AI tự động quyết định cuối cùng.

Kết nối với dự án xuyên suốt của giáo trình

- Dự án SkillGenie minh họa cho ví dụ xuyên suốt: **"Trợ lý phát triển phần mềm thông minh"**.
- Cả hai đều: phục vụ nhóm người dùng chuyên biệt; tích hợp LLM với kho tri thức chuyên ngành; đòi hỏi cơ chế đánh giá và phản hồi chất lượng cao.

Thông điệp chính

Dù bạn xây dựng trợ lý phát triển phần mềm, chatbot chăm sóc khách hàng, hay công cụ phân tích tài liệu – **quy trình và nguyên tắc là giống nhau**. Điều thay đổi là domain knowledge, dữ liệu, và các quyết định thiết kế cụ thể.

Chương 9 sẽ giúp bạn đưa ứng dụng vào vận hành thực tế với các chiến lược triển khai, giám sát và đảm bảo chất lượng trong môi trường production.

Tổng kết Section 3 – Tự kiểm tra

Những điểm cần nhớ

1. **Hai phạm trù kiểm thử**: Truyền thống (unit, integration, E2E) + Đặc thù AI (accuracy, relevance, safety, hallucination, bias).
2. **Tính không xác định** là thách thức lớn nhất. Ba chiến lược: criterion-based, LLM-as-judge, human evaluation.
3. **Hallucination**: nguyên nhân từ thiếu kiến thức, nội suy sai, áp lực tạo câu trả lời. Chiến lược hiệu quả nhất: **Grounding với RAG**.
4. **9 bước phát triển dự án** – bảo mật không phải bước cuối; con người luôn trong vòng lặp.

Tự kiểm tra nhanh:

1. Tại sao không áp dụng được test case truyền thống cho GenAI? Nêu 2 cách tiếp cận thay thế.

Tổng kết chương

Những điểm cần ghi nhớ

- **Quy trình 5 giai đoạn**: Thiết kế $\rightarrow$ Prototype $\rightarrow$ Bảo mật $\rightarrow$ Production $\rightarrow$ Tối ưu hóa, theo mô hình lặp **Build – Measure – Learn**.
- **Ba mô hình tích hợp**: Direct API Call $\rightarrow$ Function Calling $\rightarrow$ Agent-based. Nguyên tắc: luôn bắt đầu từ đơn giản nhất.
- **Kiểm thử GenAI**: Hai phạm trù (truyền thống + đặc thù AI). Thách thức: tính không xác định. Chiến lược: criterion-based, LLM-as-judge, human evaluation.
- **Hallucination & Quy trình 9 bước**: Grounding với RAG là chiến lược giảm thiểu hiệu quả nhất. Phát triển dự án theo 9 bước, bảo mật xuyên suốt, cải thiện liên tục.